

SOFTWARE DESIGN & ANALYSIS

SDA Project

Space Shooter Game

Section

BCS-4H

Instructor

Jibran Rasheed

Group Members

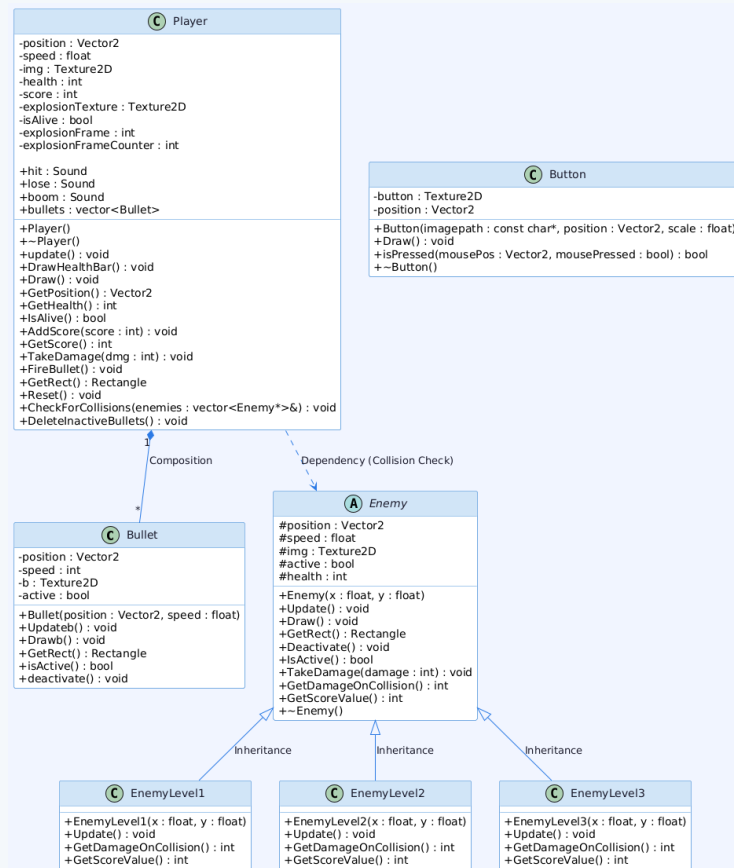
Abdullah | 24k-0859

Hammad | 24k-0544

Abdul Majid | 24k-0895

DIAGRAM 1 • UML

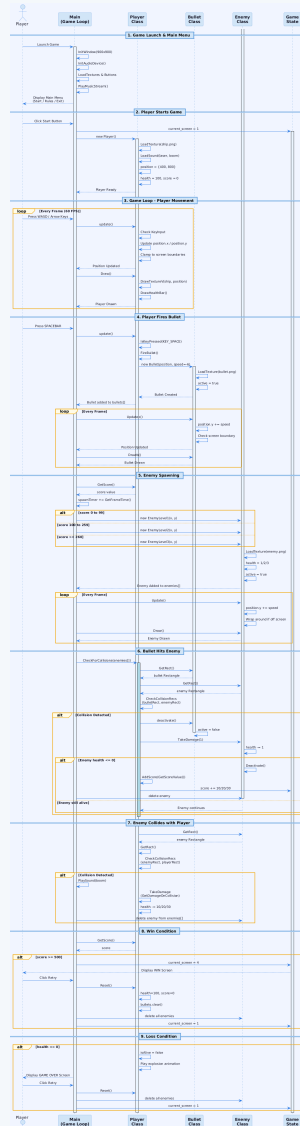
Class Diagram



The Class Diagram reveals the static object-oriented architecture of the Space Shooter game, mapping every class — Player, Enemy (and its three concrete levels), Bullet, and Button — alongside their attributes, methods, and inter-class relationships. It makes the inheritance hierarchy of EnemyLevel1/2/3 from the abstract Enemy base explicit, and demonstrates how the Player owns a collection of Bullets through composition. Unlike runtime diagrams, this view focuses purely on how the system is structured at the code level before a single frame is ever rendered.

DIAGRAM 2 · UML

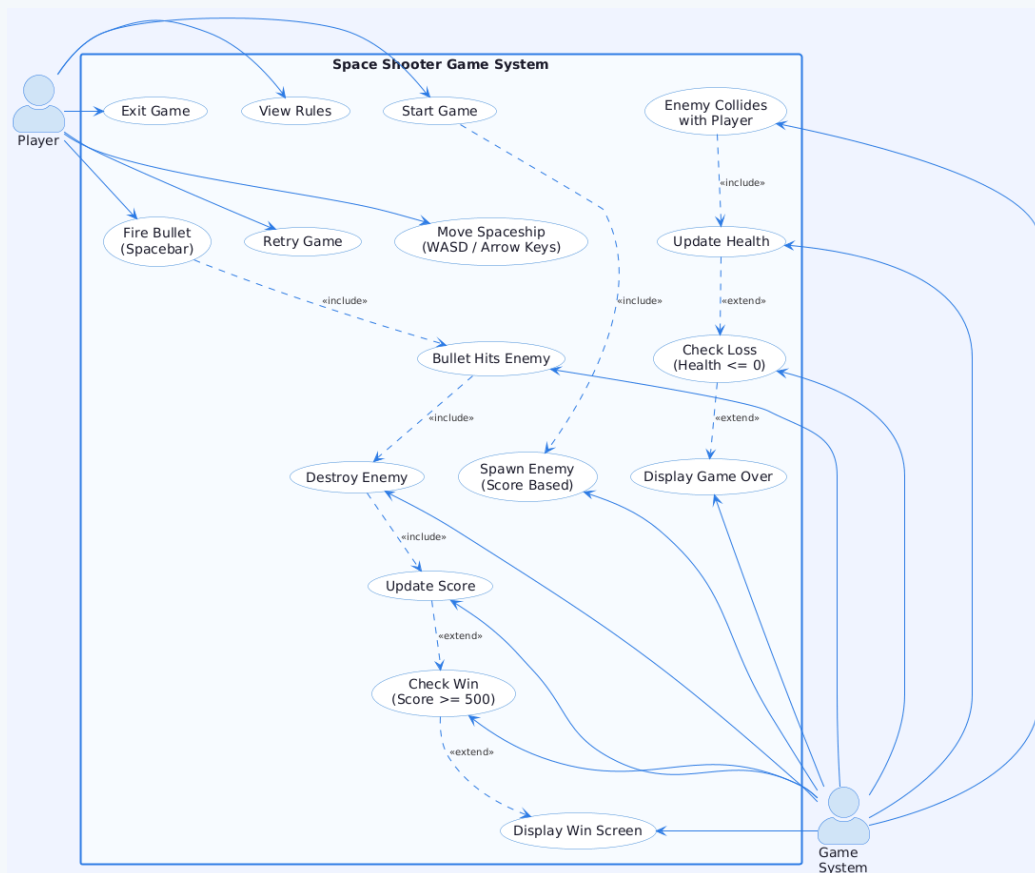
Sequence Diagram



The Sequence Diagram captures the time-ordered message flow between game objects across nine key scenarios — from window and audio initialisation, through the player firing a bullet and enemies spawning, all the way to win/loss detection and the retry flow. It makes the exact call sequence between the Game, Player, Enemy, and Bullet objects visible, showing precisely when `FireBullet()`, `CheckForCollisions()`, and `ResetGame()` are invoked. This diagram is the primary reference for understanding how components communicate at runtime.

DIAGRAM 3 • UML

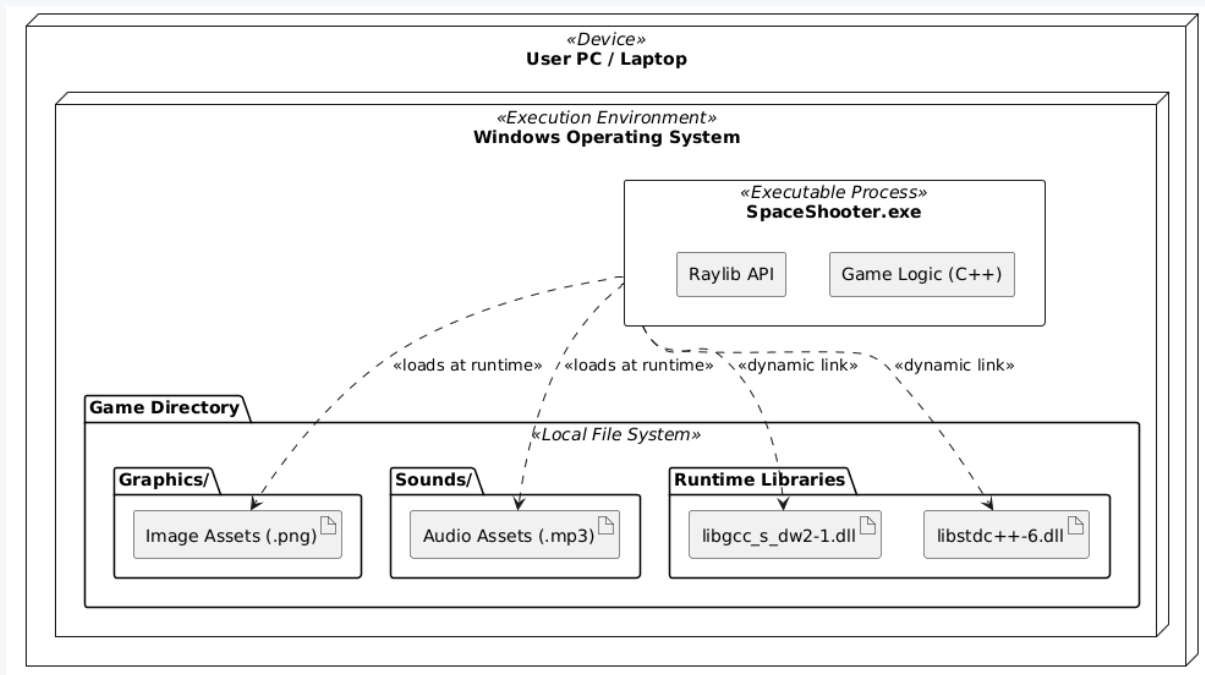
Use Case Diagram



The Use Case Diagram defines the functional scope of the game from the perspective of its two actors: the human Player and the automated Game System. Player-initiated use cases include starting the game, viewing rules, moving the spaceship, firing bullets, and retrying after a loss. System-driven use cases — enemy spawning, collision detection, score and health updates, and win/loss evaluation — run automatically every frame. The diagram also shows how Firing a Bullet always extends into collision checking, and how Score Update can trigger the Win Condition at the 500-point threshold.

DIAGRAM 4 · UML

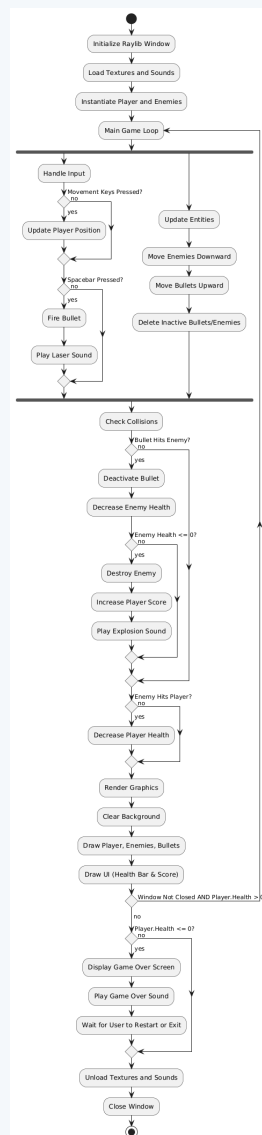
Deployment Diagram



The Deployment Diagram maps the software components and assets onto the two physical nodes involved in the game's lifecycle: the developer's machine (VSCode + MinGW + Raylib 4.5) and the player's Windows PC (OpenGL GPU + audio driver). SpaceShooter.exe is a statically-linked single binary containing all game logic, while the Graphics/ and Sounds/ folders hold the textures and audio files loaded at startup. Because Raylib is compiled directly into the executable, deployment requires nothing more than copying the game folder to any Windows computer — no installer or runtime dependency needed.

DIAGRAM 5 • UML

Activity Diagram



The Activity Diagram traces the complete execution flow of the game from startup to shutdown, breaking every frame of the main loop into six ordered phases: Input, Spawn, Update, Collision, Render, and State Check. Decision nodes at the menu (Start / Rules / Exit) and at frame-end (Win / Loss / Continue) show all branching paths through the game, including the retry reset flow. This diagram is especially critical for developers as it documents the 60-FPS game loop that drives the entire application.

ARCHITECTURAL FRAMEWORK

4+1 View Model

Logical View

Viewer: End Users

Concern: Functional requirements — what the system provides to users

UML Representation: Class Diagram

Decomposes the system into five core classes: Player (movement, shooting, health, score), abstract Enemy base, three concrete enemy levels with escalating difficulty, Bullet (self-contained projectile), and Button (reusable UI element). Inheritance, composition (Player owns vector<Bullet>), and dependency (Player references Enemy*) are all explicit in this view, making it the primary blueprint for the object-oriented design.

Process View

Viewer: System Integrators

Concern: Non-functional requirements — concurrency, performance, workflow

UML Representation: Sequence Diagram / Activity Diagram

Since the game is single-threaded, all six processes (Input, Spawn, Update, Collision, Render, State Management) execute sequentially within one main loop at 60 FPS. Enemy spawning fires every 1 second via a timer; AABB collision detection runs every frame. This view demonstrates that no threading or networking is required.

Development View

Viewer: Programmers & Software Managers

Concern: Software module organisation, layers, reuse, toolchain

UML Representation: Component / Package Diagram

main.cpp acts as controller, managing the game loop, screen states, and enemy spawning. Player, Enemy (base + 3 derived), Bullet, and Button are independent module pairs. Raylib 4.5 forms the external library layer providing graphics, audio, and input. Toolchain: C++ language, Visual Studio Code IDE, MinGW compiler, Raylib library.

Physical View

Viewer: System Engineers

Concern: Non-functional requirements regarding underlying hardware

UML Representation: Deployment Diagram

Single-node deployment: SpaceShooter.exe (statically linked Raylib) runs on a Windows PC with an OpenGL-compatible GPU and audio hardware. No DLLs or installation required — copy the folder (exe + Graphics/ + Sounds/) and run. No network, server, or cloud component exists.

Scenario / Use Case View

Viewer: All Stakeholders

Concern: *System consistency and validity — proves the design is complete*

UML Representation: *Use Case Diagram*

Ties the four architectural views together by validating them against real user scenarios: Start Game, Move Spaceship, Fire Bullet, Enemy Spawning, Collision Detection, Win/Loss Check, and Retry. Each scenario exercises classes (Logical), runtime processes (Process), modules (Development), and the deployment node (Physical), confirming the design is coherent.